7章

時間軸でモデルを作る

前の章で、複雑な業務ロジックの実装方法として、ドメインモデルを説明しました。ドメインモデルを組み立てる部品、ドメインモデルの目的、そしてドメインモデルの使いどころを学びました。この章ではイベント履歴式ドメインモデルを説明します。基本となる考え方はドメインモデルと同じです。イベント履歴式ドメインモデルも、複雑な業務ロジックと中核の業務領域を対象にします。さらに言えば、モデルを構成する基本部品も同じです。つまり、値オブジェクト、集約そして業務イベントです。

二つが異なるのは、集約の状態を永続化する方法です。イベント履歴式ドメインモデルでは、イベントソーシングを使って集約の状態を管理します。つまり、集約の状態を永続化するのではなく、集約の状態が変化したことを業務イベントで表現し、業務イベントの履歴を「真実を語る唯一の情報源」（the source of truth）として永続化します。

この章では、最初にイベントソーシングという考え方を紹介します。次に、イベントソーシングとドメインモデルを組み合わせた、イベント履歴式ドメインモデル（event-sourced domain model）について説明します。

7.1　イベントソーシング

フローチャートを見てもテーブルを見ないと何もわかりません。テーブルのデータを見ればフローチャートを見る必要はありません。

——フレッド・ブルックス[＊1]

[＊1]　F. P. Brooks Jr著『The Mythical Man-Month: Essays on Software Engineering』（Addison-Wesley、1974年）（邦訳『人月の神話』ピアソンエデュケーション、2002年）

イベントソーシングがどのようなもので、従来のモデリングやデータの永続化と何が違うかを理解するために、このフレッド・ブルックスの考え方を使ってみましょう。表7‐1のデータから、何がわかるでしょうか。

表7-1　状態を元にしたモデル

lead-id	last-name	first-name	sta-tus	phone-num-ber	followup-on	created-on	updated-o
1	中島	瑞希	成約済	555-1246		2019-01-31T 10:02:40.32Z	2019-01-3 10:02:40.3
2	神田	謙二	終了	555-4395		2019-03-29T 22:01:41.44Z	2019-03-2 22:01:41.4
3	中村	喜彦	終了	555-1176		2019-04-15T 23:08:45.59Z	2019-04-1 23:08:45.5
4	麓	孝敏	終了	555-1850		2019-04-25T 05:42:17.07Z	2019-04-2 05:42:17.0
5	田淵	貴行	成約済	555-3013		2019-05-14T 04:43:57.51Z	2019-05-1 04:43:57.5
6	飯田	司	新規	555-2900		2019-06-19T 15:01:49.68Z	2019-06-1 15:01:49.6
7	山本	真由香	新規	555-8861		2019-07-17T 13:09:59.32Z	2019-07-1 13:09:59.3
8	浅野	麻衣	終了	555-3018		2019-10-09T 11:40:17.13Z	2019-10-0 11:40:17.1
9	尾崎	智子	商談	555-6461	2019-12-04T 01:49:08.05Z	2019-12-04T 01:49:08.05Z	2019-12-0 01:49:08.0

lead-id	last-name	first-name	sta-tus	phone-num-ber	followup-on	created-on	updated-o
			予定				
10	矢田	雅也	終了	555-4090		2019-12-06T 09:12:32.56Z	2019-12-0 09:12:32.5
11	井本	泰之	商談予定	555-1187	2020-01-23T 00:33:13.88Z	2020-01-23T 00:33:13.88Z	2020-01-2 00:33:13.8
12	小林	裕美	成約済	555-8101		2020-05-20T 09:52:55.95Z	2020-05-2 12:38:44.1
13	小林	俊介	新規	555-4753		2020-04-20T 06:52:55.95Z	2020-04-2 06:52:55.9
14	井上	直人	成約済	555-1284		2020-05-06T 18:47:04.70Z	2020-05-0 18:47:04.7
15	桑山	祐太	支払なし	555-3230		2020-06-04T 14:51:06.15Z	2020-06-0 14:51:06.1
16	草間	三枝	新規	555-6953		2020-06-09T 09:07:05.23Z	2020-06-0 09:07:05.2
17	星谷	貴子	成約済	555-2326		2020-07-20T 00:56:59.94Z	2020-07-2 00:56:59.9
18	磯部	麻子	支払	555-2620		2020-08-12T 17:39:43.25Z	2020-08-1 17:39:43.2

lead-id	last-name	first-name	sta-tus	phone-num-ber	followup-on	created-on	updated-o
			待ち				
19	清水	直子	商談予定	555-8163	2020-10-22T 12:40:03.98Z	2020-10-22T 12:40:03.98Z	2020-10-2 12:40:03.9
20	林原	学	支払待ち	555-9273		2020-11-13T 08:14:07.17Z	2020-11-1 08:14:07.1

データ内容を見ればわかるように、これは販売管理システムの見込み客に関するデータです。見込み客ごとに、管理番号、姓名、レコードの作成日時と更新日時、電話番号、そして商談の進捗状況を表現しています。

商談の進捗状況を分析すると、見込み客への商談活動のプロセスは次のように想定できそうです。

- 商談は新規からスタートする
- 見込み客への商談の結果は、断られた（終了）、後日連絡することになった（商談予定）、商談が成立した（支払待ち）のいずれかになる
- 支払が完了すると成約済になる。支払が行われなかった場合は支払なしになる

テーブルのカラム構成と実際のデータを調べると、これだけの情報が手に入ります。そしてこのデータモデルは同じ言葉を使って作られたと想像できます。しかし欠落している情報があります。何が欠落しているかわかりますか。

このテーブルが表現しているのは、見込み客の「現在」の状態です。しかし、どのような経緯で現在の状態に到達したのかがわかりません。成約済になるまでに、電話を何回かけたでしょうか。すんなりと買ってもらえたのでしょうか。それともかなりのやりとりを繰り返したのでしょうか。商談の履歴から判断して、この見込み客と商談を続ける価値があるでしょうか。それとも、こ

の見込み客との商談は終わりにして、もっと有望な見込み客との商談にとりかかったほうがよいのでしょうか。 このテーブルのデータには、こういう問いに答えるための情報が欠けています。このデータからわかるのは商談の現在の状態だけです。

このような問いかけは、商談プロセスを最適化したいという、事業活動の本質的な関心事から生まれます。商談の記録を分析し、事実にもとづいて商談プロセスを最適化することは、事業活動にとって、きわめて重要な取り組みです。不足しているこのような情報を集める方法の一つがイベントソーシングです。

イベントソーシングは、時間軸の視点を取り入れたデータモデルです。イベントソーシングを取り入れたシステムでは、集約の現在の状態を記録するかわりに、集約のライフサイクルで発生するすべての変化を一連のイベントとして永続化します。

表7‐1の12行目の成約済の顧客について考えてみましょう。イベントソーシングでは、見込み客のデータを次のようなリスト形式で表現します。

```
{
  "lead-id": 12,
  "event-id": 0,
  "event-type": "新規登録",
  "last-name": "小林",
  "first-name": "浩美",
  "phone-number": "555-2951",
  "timestamp": "2020-05-20T09:52:55.95Z"
},
{
  "lead-id": 12,
  "event-id": 1,
  "event-type": "架電",
  "timestamp": "2020-05-20T12:32:08.24Z"
},
{
  "lead-id": 12,
  "event-id": 2,
  "event-type": "商談予定設定",
  "followup-on": "2020-05-27T12:00:00.00Z",
  "timestamp": "2020-05-20T12:32:08.24Z"
},
{
```

```
    "lead-id": 12,
    "event-id": 3,
    "event-type": "連絡先変更",
    "last-name": "小林",
    "first-name": "裕美",
    "phone-number": "555-8101",
    "timestamp": "2020-05-20T12:32:08.24Z"
  },
  {
    "lead-id": 12,
    "event-id": 4,
    "event-type": "架電",
    "timestamp": "2020-05-27T12:02:12.51Z"
  },
  {
    "lead-id": 12,
    "event-id": 5,
    "event-type": "注文受領",
    "payment-deadline": "2020-05-30T12:02:12.51Z",
    "timestamp": "2020-05-27T12:02:12.51Z"
  },
  {
    "lead-id": 12,
    "event-id": 6,
    "event-type": "支払完了",
    "status": "成約",
    "timestamp": "2020-05-27T12:38:44.12Z"
  }
```

このような一連のイベント記録があれば、顧客が成約済になるまでのストーリーがわかります。見込み客を新規に登録し（イベント番号0）、その2時間後には商談担当者が顧客に架電（電話で連絡）しています（イベント番号1）。その会話の中で1週間後にまた連絡することになりました（イベント番号2）。連絡先を別の電話番号に変更し（イベント番号3）、氏名も訂正しました（イベント番号3）。約束通りにもう一度、架電したところ（イベント番号4）、注文してもらえました（イベント番号5）。支払は3日以内という約束をしました（イベント番号5）。実際の支払は30分後に行われ、契約が成立しました（イベント番号6）。この結果、見込み客は新規顧客になりました。

この業務イベントの履歴を使えば、顧客の最新状態を簡単に投影できます。次のように、それぞれのイベントに対して単純な変換ロジックを順次実行すればよいのです。

```
//見込み客状態モデルの投影
public class LeadStateModelProjection
{
  public long LeadId { get; private set; }
  public string FirstName { get; private set; }
  public string LastName { get; private set; }
  public LeadStatus Status { get; private set; }
  public PhoneNumber PhoneNumber { get; private set; }
  public DateTime? FollowupOn { get; private set; }
  public DateTime CreatedOn { get; private set; }
  public DateTime UpdatedOn { get; private set; }
  public int Version { get; private set; }

  // 新規登録イベント
  // @eventは、予約語のeventを変数名として使うためのC#の記法
  public void Apply(LeadInitialized @event)
  {
    LeadId = @event.LeadId;
    Status = LeadStatus.NEW_LEAD;
    FirstName = @event.FirstName;
    LastName = @event.LastName;
    PhoneNumber = @event.PhoneNumber;
    CreatedOn = @event.Timestamp;
    UpdatedOn = @event.Timestamp;
    Version = 0;
  }

  // 架電イベント
  public void Apply(Contacted @event)
  {
      UpdatedOn = @event.Timestamp;
      FollowupOn = null;
      Version += 1;
  }

  // 商談予定設定イベント
  public void Apply(FollowupSet @event)
  {
      UpdatedOn = @event.Timestamp;
      FollowupOn = @event.FollowupOn;
      Status = LeadStatus.FOLLOWUP_SET;
      Version += 1;
  }

  // 連絡先変更イベント
  public void Apply(ContactDetailsChanged @event)
  {
```

```
        FirstName = @event.FirstName;
        LastName = @event.LastName;
        PhoneNumber = @event.PhoneNumber;
        UpdatedOn = @event.Timestamp;
        Version += 1;
    }

    // 注文受領イベント
    public void Apply(OrderSubmitted @event)
    {
        UpdatedOn = @event.Timestamp;
        Status = LeadStatus.PENDING_PAYMENT;
        Version += 1;
    }

    // 支払完了イベント
    public void Apply(PaymentConfirmed @event)
    {
        UpdatedOn = @event.Timestamp;
        Status = LeadStatus.CONVERTED;
        Version += 1;
    }
  }
```

記録されているすべての業務イベントに対して、イベントの種類ごとに多重定義した、`Apply`メソッドを順番に実行します。その結果、表7‐1のレコード内容（管理番号、姓名、電話番号）を正しく再現できます。

`Version`フィールドに注目してください。業務イベントを適用するたびにバージョン番号を増やしています。バージョン番号はエンティティの状態を変更した回数です。さらに言えば、業務イベントを部分的に適用することで「タイムトラベル」ができます。つまり、ある時点までのイベントを適用して、その時点の状態を投影できます。たとえば、バージョン5の状態を知りたければ、最初の五つのイベントだけ適用すればよいのです。

このやり方で投影できるのは特定時点の状態だけではありません。いくつかのシナリオを考えてみましょう。

7.1.1　検索

検索機能を考えてみましょう。見込み客の連絡先、つまり姓名と電話番号は変更可能です。ですから、ある担当者が連絡先を変更した時に、別の担当者はその変更に気がつかず、変更前の姓名や電話番号で検索するかもしれません。こういう変更履歴まで含めた情報も、次のやり方で簡単に投影できます。

```
// 見込み客検索モデルの投影
public class LeadSearchModelProjection
{
  public long LeadId { get; private set; }
  public HashSet<string> FirstNames { get; private set; }
  public HashSet<string> LastNames { get; private set; }
  public HashSet<PhoneNumber> PhoneNumbers { get; private set; }
  public int Version { get; private set; }

  // 新規登録イベント
  public void Apply(LeadInitialized @event)
  {
    LeadId = @event.LeadId;
    FirstNames = new HashSet<string>();
    LastNames = new HashSet<string>();
    PhoneNumbers = new HashSet<PhoneNumber>();

    FirstNames.Add(@event.FirstName);
    LastNames.Add(@event.LastName);
    PhoneNumbers.Add(@event.PhoneNumber);

    Version = 0;
  }

  // 連絡先変更イベント
  public void Apply(ContactDetailsChanged @event)
  {
    FirstNames.Add(@event.FirstName);
    LastNames.Add(@event.LastName);
    PhoneNumbers.Add(@event.PhoneNumber);

    Version += 1;
  }

  // 架電イベント
  public void Apply(Contacted @event)
  {
    Version += 1;
  }

  // 商談予定設定イベント
  public void Apply(FollowupSet @event)
  {
    Version += 1;
  }

  // 注文受領イベント
```

```
    public void Apply(OrderSubmitted @event)
    {
      Version += 1;
    }

    // 支払完了イベント
    public void Apply(PaymentConfirmed @event)
    {
      Version += 1;
    }
  }
```

この投影ロジックは、`LeadInitialized`イベントと`ContactDetailChanged`イベントを使って、それぞれのイベントで発生した連絡先情報を生成します。他のイベントは連絡先情報を変更しません。

この投影ロジックを小林裕美さんのイベント履歴に適用すると、変更前の情報も含んだ以下の内容を投影できます。

```
LeadId: 12
LastNames: ['小林']
FirstNames: ['浩美', '裕美']
PhoneNumbers: ['555-2951', '555-8101']
Version: 6
```

7.1.2　分析

ビジネス分析チームから、見込み客についてもっと分析に適した形式のデータがほしいと言われました。分析チームが知りたいのは、見込み客へのフォローアップ電話の回数です。そして成約までこぎつけたか、あるいは途中で終了したかの結果と突き合わせて、効果的な商談活動のやり方を見つけようとしています。分析チームが必要としているデータを、次の方法で投影してみましょう。

```
// 分析モデルの投影
public class AnalysisModelProjection
{
  public long LeadId { get; private set; }
  public int Followups { get; private set; }  // 商談予定設定回数
  public LeadStatus Status { get; private set; }
  public int Version { get; private set; }

  // 新規登録イベント
```

```
    public void Apply(LeadInitialized @event)
    {
      LeadId = @event.LeadId;
      Followups = 0;
      Status = LeadStatus.NEW_LEAD;
      Version = 0;
    }

    // 架電イベント
    public void Apply(Contacted @event)
    {
      Version += 1;
    }

    // 商談予定設定イベント
    public void Apply(FollowupSet @event)
    {
      Status = LeadStatus.FOLLOWUP_SET;
      Followups += 1;
      Version += 1;
    }

    // 連絡先変更イベント
    public void Apply(ContactDetailsChanged @event)
    {
      Version += 1;
    }

    // 注文受領イベント
    public void Apply(OrderSubmitted @event)
    {
      Status = LeadStatus.PENDING_PAYMENT;
      Version += 1;
    }

    // 支払完了イベント
    public void Apply(PaymentConfirmed @event)
    {
      Status = LeadStatus.CONVERTED;
      Version += 1;
    }
  }
```

商談予定設定イベントを適用するロジックで、予定した回数（`Followups`）を数えるようにしました。この投影ロジックを使えば、既存のイベント履歴から、以下の情報を生成できます。

```
LeadId: 12
Followups: 1
Status: 成約
Version: 6
```

ここまでのコード例は、検索に最適化したモデルや分析に最適化したモデルをメモリ上で投影しています。しかし、この機能を実際に使えるようにするには、投影したモデルをデータベースに永続化する必要があります。次の**第8章**で、この具体的なやり方、つまりコマンド・クエリ責任分離（CQRS）を学びます。

7.1.3　真実を語る唯一の情報源

イベントソーシングを実現するには、オブジェクトのすべての状態変更をイベントとして表現し、永続化することが必要です。永続化されたイベントが「真実を語る唯一の情報源」（source of truth）となります。つまりイベント（event）ソーシング（sourcing）です。このやり方は**図7‐1**のようになります。

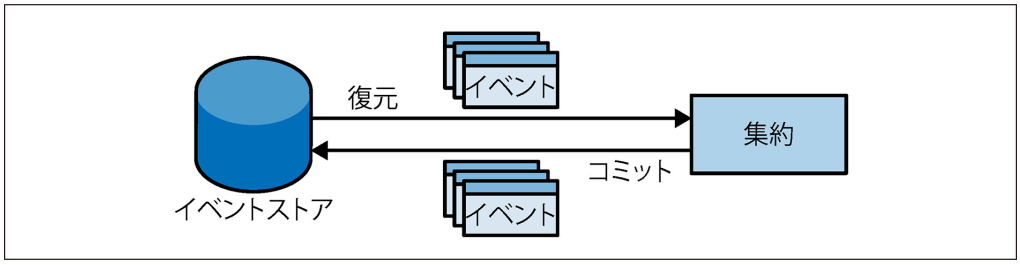

図7-1　イベント履歴式の集約

データの一貫性が保証されるのは、つまり真実を語る唯一の情報源は、システムのイベントを記録したデータベースだけです。一連のイベントを永続化するこのデータベースをイベントストアと呼びます。

7.1.4　イベントストア

イベントストアは追記専用です。記録したイベントの変更や削除はできません**＊2**。イベントソーシングを実現するために、イベントストアで最低限必要な機能は次の二つです。

＊2　データ移行など特殊な場合を除きます。

- 特定のエンティティに関するすべてのイベントを取り出す

- イベントを追記する

コードで表現すると次のようになります。

```
interface IEventStore
{
  IEnumerable<Event> Fetch(Guid instanceId);
  void Append(Guid instanceId, Event[] newEvents, int expectedVersion);
}
```

`Append`メソッドの第3引数`expectedVersion`は楽観的な排他制御を実現するために必要です。つまり、新たなイベントを追記する時に、バージョンを引数に指定することで、競合の発生を検知します。もし指定したバージョンがデータストア側の最新バージョンよりも古い、つまり別のプロセスがイベントを追加済みであれば、イベントストアは書き込みを失敗させ例外を送出します。

ほとんどのシステムはこのようなCQRSのやり方を実装するためには、追加のエンドポイントが必要です。CQRSの実現方法は次の章で説明します。

本質的には、イベントソーシングは新しい考え方ではありません。会計の世界では、元帳の変化を表現するためにイベントを使います。元帳は会計取引を追記型で記録します。現在の状態、つまり口座の残高は、元帳の記録からいつでも「投影」できます。

7.2　イベント履歴式ドメインモデル

前の章で説明したドメインモデルでは、集約は最新の状態を表現し、特定の業務イベントを発信します。それに対し、イベント履歴式ドメインモデルでは、集約のライフサイクルを業務イベントでモデル化します。集約のすべての状態変更を業務イベントで表現することが必要です。

イベント履歴式集約への操作は以下の手順で行います。

1. 業務イベント履歴を読み込む
2. 業務判断の対象となる、ある時点の状態を業務イベント履歴から再構築する

3. 集約のコマンド（業務ロジック）を実行し、その結果として、業務イベントを生成する
4. 新たに生成した業務イベントをイベントストアにコミットする

第6章で説明した`Ticket`集約をイベント履歴式集約で実装してみます。

まずアプリケーションサービスで、この四つの手順を実装します。つまり、チケットに発生した一連のイベントを取得し、集約を再構築し、適切なコマンドを実行し、状態の変化をデータベースに永続化します。コードは次のようになります。

```
01 public class TicketAPI
02 {
03   private ITicketsRepository _ticketsRepository;  // イベントストア
04   ...
05
06   public void RequestEscalation(TicketId id, EscalationReason reason)
07   {
08     var events = _ticketsRepository.LoadEvents(id);
09     var ticket = new Ticket(events);
10     var originalVersion = ticket.Version;
11     var cmd = new RequestEscalation(reason);
12     ticket.Execute(cmd);
13     _ticketsRepository.CommitChanges(ticket, originalVersion);
14   }
15
16   ...
17 }
```

このアプリケーションサービスが使う`Ticket`集約の実装例は以下のとおりです。最新の状態を投影するために、コンストラクタ（27行目から31行目）で`TicketState`クラスを生成し、すべての業務イベントに対して順番に`AppendEvent`メソッドを実行します。

```
18 public class Ticket
19 {
20   ...
21   private List<DomainEvent> _domainEvents = new List<DomainEvent>();
22   private TicketState _state;
23   ...
24
25   public Ticket(IEnumerable<IDomainEvents> events)
26   {
```

```
27      _state = new TicketState();
28      foreach (var e in events)
29      {
30        AppendEvent(e);
31      }
32    }
```

`AppendEvent` メソッドは、以下のように、到着した業務イベントを `TicketState` クラスの投影ロジックに渡し、チケットの最新状態をメモリ上に構築します。

```
33    private void AppendEvent(IDomainEvent @event)
34    {
35      _domainEvents.Append(@event);
36      // TicketStateクラスに多重定義された適切なApplyメソッドが動的に呼び出され
37      ((dynamic)_state).Apply((dynamic)@event);
38    }
```

第6章の `Ticket` クラスでは、 `RequestEscalation` コマンドを受け取ると、 `IsEscalated` フラグを `True` に設定していました。それに対し、イベント履歴式集約では、次のように、 `escalatedEvent` を生成して `AppendEvent` メソッドに渡します（43行目と44行目）。

```
39    public void Execute(RequestEscalation cmd)
40    {
41      if (!_state.IsEscalated && _state.RemainingTimePercentage <= 0)
42      {
43        var escalatedEvent = new TicketEscalated(_id, cmd.Reason);
44        AppendEvent(escalatedEvent);
45      }
46    }
47
48    ...
49  }
```

`Ticket` 集約内のイベント履歴 `_domainEvents` に追加されたすべてのイベントは、 `TicketState` クラスの投影ロジックである `Apply` メソッドに渡されます。 `TicketState` クラスのフィールドは次のようにイベント内容に応じて変更されます。

```
public class TicketState
{
  public TicketId Id { get; private set; }
  public int Version { get; private set; }
  public bool IsEscalated { get; private set; }
  ...
  public void Apply(TicketInitialized @event) // 新規登録
  {
    Id = @event.Id;
    Version = 0;
    IsEscalated = false;
    ....
  }

  public void Apply(TicketEscalated @event) // チケットエスカレーション
  {
    IsEscalated = true;
    Version += 1;
  }

  ...
}
```

　では、複雑な業務ロジックを実装する方法として、イベントソーシングにどんなメリットがあるか見てみましょう。

「イベント履歴式ドメインモデル」という用語にした理由

「イベントソーシング」ではなく「イベント履歴式ドメインモデル」という用語にした理由を説明させてください。状態の遷移をイベントで表現するイベントソーシングは、さまざまな状態管理に適用可能です。イベントソーシングの適用対象は、ドメインモデルの部品である集約の状態管理に限定されません。そのため、集約のライフサイクルの状態管理にイベントソーシングを使っていることを明示するためには、イベント履歴式ドメインモデルという長い用語のほうが好ましいと考えました。

7.2.1　利点

従来のやり方では、集約の最新状態をデータベースに記録します。それに比べ、イベント履歴式ドメインモデルを選択した場合、集約の設計はたいへんになります。しかし、この方法は重要な利点があります。次のような場面でこのやり方を検討する価値があります。

タイムトラベル

業務イベントの履歴から、集約の最新状態を構築するのと同じ方法で、過去のあらゆる時点の状態を再現できます。別の言い方をすれば、集約のあらゆる過去の状態を、任意に再現できます。

過去の再現は、システムの使われ方の分析、システムが行った判定結果の検査、そして業務ロジックの最適化などに使えます。

また障害や不具合の調査にも活用できます。つまり、障害が発生した時点の集約の状態を正確に再現できます。

深い洞察

第Ⅰ部で、中核の業務領域を最適化することは、事業活動において戦略的に重要であることを見てきました。イベントソーシングによって、システムの状態と使われ方を深く洞察できるようになります。この章で学んできたように、イベントソーシングは、業務イベントの履歴という一つの情報源から、さまざまな状態を表現できます。つまり、状態を投影する新たなロジックをいつでも追加できるので、既存のイベント履歴から、業務活動について新たな洞察を手に入れることが可能です。

監査ログ

業務イベントの履歴は厳密な一貫性を持つ監査ログです。集約の状態を変化させたすべての業務活動の正確な記録です。事業領域によっては、このような監査ログの保存が法的に義務づけられています。イベントソーシングを採用すればこの要求に簡単に対応できます。

監査ログ方式は、金銭や金融取引を管理するシステムで威力を発揮します。どのような計算判断が行われ、どのように資金が移動したかを簡単に追跡できます。

進化した楽観的排他制御

従来の楽観的な排他制御の仕組みでは、読み込んだデータを変更して書き込む途中で他のプロセスが同じデータを上書きしたことを検知すると、例外が発生

し、書き込みは失敗します。

イベントソーシングを採用すると、既存のイベントを読み込んでから新しいイベントを書き込むまでに起きることを正確に把握できます。イベントストアに同時に追加された業務イベントを取得し、イベントを追加する操作が衝突しているのか、あるいは、それぞれの業務イベントは無関係であり、そのまま操作を継続しても安全かどうかを業務視点で判定するロジックを組み込めます。

7.2.2　欠点

今までの説明だけ見れば、イベント履歴式ドメインモデルは、業務ロジックを実装する究極のやり方であり、できるだけこの方法を使うべきだと思うかもしれません。もちろん、こういう単純な見方は「事業活動の視点で設計判断する」という原則から外れます。イベント履歴式ドメインモデルの以下の課題を検討してみましょう。

学習曲線

このやり方の明らかな欠点は、従来のデータ管理の方法とまったく違うやり方であることです。このやり方を正しく実装するにはチームの訓練が必要です。新しい考え方に慣れるまでには、そうとうな時間がかかります。チームがイベント履歴式のシステムを開発した経験がない限り、この学習曲線を考慮に入れなければなりません。

モデルの発展性

イベント履歴式ドメインモデルを進化させることは、たいへんかもしれません。イベントソーシングの厳密な定義では、イベントは不変（イミュータブル）です。しかし、イベントのデータ構造を変えたい場合、どうすればよいでしょうか。いちど定義したイベントのデータ構造を変えるプロセスは、テーブル設計の変更に比べ、はるかに難しい課題です。実際、グレッグ・ヤングはこのテーマだけで本を1冊書いています**＊3**。

＊3　Greg Young著『Versioning in an Event Sourced System』（**https://leanpub.com/esversioning**）

技術方式の複雑さ

イベントソーシングの実装は、システムにさまざまな仕組みを持ち込むことになり、システム全体の設計が複雑になります。この点は、次の章でCQRSを検討する時に詳しく取り上げます。

目の前の課題解決にイベント履歴式ドメインモデルを使う正当な理由がなく、もっと単純な解決方法を採用できる場合、ここで説明した欠点は深刻な問題となります。**第10章**で、業務ロジックの実装方法をどうやって選択すればよいかを学びます。

7.3　よくある質問

イベントソーシングを初めて知った技術者は、たいてい似たような疑問を持ちます。そういうよくある質問にここで回答しておきしょう。

7.3.1　性能

イベント履歴から状態を再構築するこのやり方は、性能面が気になります。イベントが増えると、性能がかなり悪くなりそうですが、この方法でうまくいくのでしょうか。

イベント履歴から状態を投影するには、それなりの処理能力は必要です。そして、集約のイベント履歴が大きくなるほど性能が劣化します。

状態投影が性能にどのくらい影響するかは、実際に計測してみることが重要です。つまり、数百あるいは数千のイベント履歴の処理がどのくらい性能に影響するかの測定です。その測定結果と集約の現実的な大きさ、つまり一つの集約に対して発生が予想されるイベント数、を比較して検討してください。

ほとんどのシステムでは、一つの集約のイベントが一万を超えれば性能に影響がでるでしょう。実際には、集約のイベント数が百を超えるようなシステムはほとんどないでしょう。

性能に影響がでるほど巨大なイベント履歴を扱う必要がある特殊な状況では、別のやり方としてスナップショットがあります。スナップショットの実現方法は次のとおりです（**図7‐2**）。

- 独立したプロセスがイベントストアを監視し、新たなイベントを検知したら、最新の状態を投影してキャッシュに書き込む
- 集約に対する操作を実行するために、メモリー上に最新状態を投影する場合、以下を行う
  - 投影済の状態（スナップショット）をキャッシュから取得する
  - スナップショットを作った時点より後に追加されたイベントをイベントストアから取得する
  - 取得したイベントをメモリー上でスナップショットに適用する

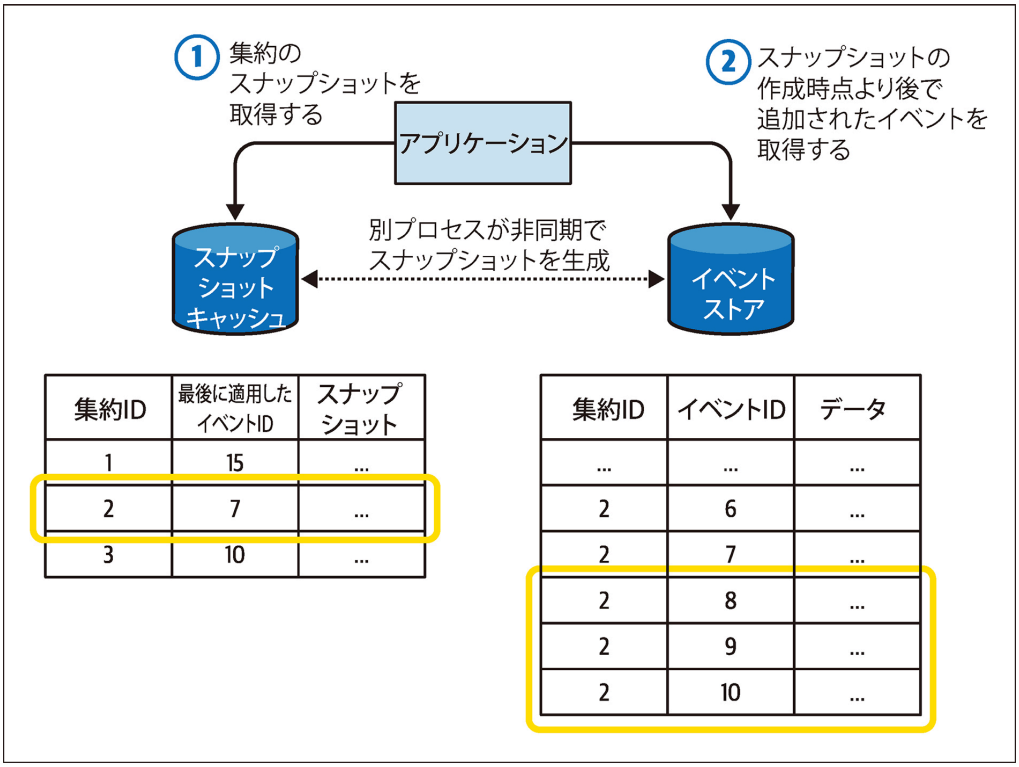

図7-2　集約のイベント履歴のスナップショット

繰り返しになりますが、スナップショットを採用することが最適かどうかは慎重に判断すべきです。集約の一つのインスタンスに必要なイベント履歴が一万を超えるような規模でもない限り、スナップショット方式の複雑さは必要ありません。スナップショット方式が適切と思える場合でも、いちど立ち止まり、集約の境界の設計を見直すことをおすすめします。

このやり方だと大量のデータが発生しそうです。スケールできますか。

イベント履歴式ドメインモデルは簡単にスケールできます。集約に関する操作は、集約のインスタンスごとに実行されます。ですからイベントストアは集約IDでシャーディング（水平分割）できます。つまり、特定のインスタンスに属するすべてのイベントを同じシャードに書き込みます（図7‐3）。

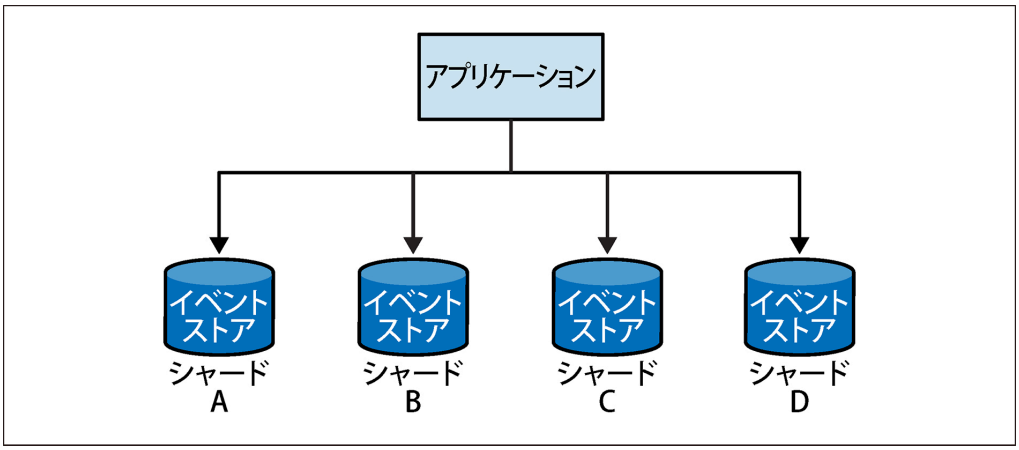

図7-3　イベントストアの分割（シャーディング）

7.3.2　データの削除

イベントストアは追記専用のデータベースですが、データの物理削除が必要になった場合、どうすればよいでしょうか。たとえば、GDPR（EU一般データ保護規則）＊4に適合するためです。

＊4　GDPR：General Data Protection Regulation（https://oreil.ly/08px7）（2021年7月14日参照）

この問題は、イベントソーシングの設計パターンの一つである、「忘れられる内容」（forgettable payload）で解決します。まず、イベントに含まれる、慎重に扱うことが必要なすべての情報を暗号化します。この暗号化のキーはイベントストアとは別のKey-Valueストア保存します。Keyは集約の識別番号です。Valueは暗号化のキーです。暗号化された情報を削除したい場合、Key-Valueストアに保存した暗号化のキーを削除します。その結果、イベントに含まれる保護すべき情報は復元できなくなります。

7.3.3　他のやり方ではだめですか？

ログをテキストファイルに書き出して、それを監査ログとして使う方法ではだめですか。

データを業務系データベースとログファイルの両方に書き込むやり方は、まちがいを起こしやすいやり方です。本質的に異なる二つの記録方法、つまりデータベースへの書き込みとファイルへの書き込みの、トランザクション管理の問題です。もし片方の書き込みが失敗したら、もう片方の書き込みをロールバックする必要がありますが、たとえば、データベーストランザクションが失敗しても、先行して書かれたログファイルの記録は削除されません。したがって、このようなログは一貫性がなく、結果的に不整合（eventually inconsistent）となります。

状態ありのやり方、つまり、状態の書き込みと履歴テーブルへの書き込みを一つのデータベーストランザクションで処理する方法ではだめですか。

技術的な観点からは、この方法は状態と履歴の一貫性を実現しています。しかし、このやり方もまちがいを起こしやすい方法です。たとえば、将来プログラムの変更が必要になった時、保守担当の技術者が履歴テーブルに反映することを忘れてしまえばどうなるでしょう。

もっというと、状態変更ありのテーブルを「真実を語る唯一の情報源」とした場合、脇役となる履歴テーブルの扱いは、いいかげんになりがちです。履歴テーブルで情報が欠落し不適切な形式の書き込みがされることを防ぐ方法はありません。

状態を記録するテーブルにトリガーを追加して、すべての項目のスナップショットを履歴テーブルに複製する方法ではだめですか。

トリガーによる履歴の書き込みは、前の方法の欠点を克服できます。つまり、履歴テーブルへの書き忘れは起きません。とはいえ、この方法は単なるデータの記録、つまり項目の変化内容が記録されるだけです。このやり方では業務の文脈が欠落します。つまり、その項目が変更された理由を記録できません。理由がわからなければ、新たなモデルを投影できる可能性は、きわめて小さくなります。

7.4　まとめ

この章ではイベントソーシングを説明しました。そして、その応用として、ドメインモデルの集約を時間軸の視点でモデル化する方法を説明しました。

イベント履歴式ドメインモデルでは、集約の状態変化を一連の業務イベントで表現します。従来の方法、つまり集約の変更後の最新状態をデータベースに永続化する方法とは大きく異なります。集約の最新状態は業務イベントの履歴から投影します。さらに、業務ロジックの実装方法をイベント履歴式モデルにすることで、目的に合わせたさまざまなモデルを柔軟に投影できます。

このやり方は業務データから深い洞察を得るためにきわめて重要です。業務活動の分析と最適化、あるいは法律が定める監査記録などに活用できます。

第5章、**第6章**、そしてこの第7章までは、業務ロジックを対象にしたモデル化と実装の方法を探求してきました。つまり、トランザクションスクリプト、アクティブレコード、ドメインモデルそしてイベント履歴式ドメインモデルの四つの実装方法です。次の章は、検討の対象範囲を広げて、アプリケーション全体を組み立てる方法（技術方式）を見ていきます。

7.5　演習問題

1. 業務イベントと値オブジェクトの関係の説明として正しいのはどれですか？
   A. 業務イベントは値オブジェクトを使って業務活動で起きた事実を記述する

B. イベント履歴式ドメインモデルを実装するには、値オブジェクトをイベント履歴式の集約に変える必要がある
C. 値オブジェクトは従来のドメインモデル方式に関係する。イベント履歴式ドメインモデルでは、値オブジェクトのかわりに業務イベントを使う
D. どの説明もまちがっている

2. 業務イベントの履歴から状態を投影するやり方の説明として正しいのはどれですか？
A. イベント履歴から投影できる状態は一つだけである
B. イベント履歴からさまざまな切り口の状態を投影できるが、そのためには、さまざまな状態の投影ができるように業務イベントを設計する必要がある
C. イベント履歴からさまざまな切り口の状態を投影できる。また後から新たな切り口の投影を追加していける
D. どの説明もまちがっている

3. 状態変更ありの集約とイベント履歴式の集約の違いを正しく説明しているのはどれですか？
A. イベント履歴式の集約は業務イベントを生成できるが、状態変更ありの集約では業務イベントは生成できない
B. どちらの方式でも業務イベントを生成できるが、業務イベントを「真実を語る唯一の情報源」とするのはイベント履歴式の集約だけである
C. イベント履歴式の集約は、すべての状態遷移に対して業務イベントを生成する
D. BとCが正しい

4. 「この本について」で紹介したウルフデスク社の例で、イベント履歴式ドメインモデルが役に立つ機能はどれでしょう？